

Prompting BP

Cursor AI 실습 회고 & 현업 적용 — VSCode C++ & Java 개발자 과정

1

실습 회고

KPT 구조화 회고 + 개인 학습 성과

2

Prompting BP 사례

C++ & Java 실전 우수 프롬프트 패턴

3

팀 액션 아이템

SMART 목표 + 현업 적용 계획

4

현업 적용 로드맵

1개월·3개월·6개월 단계별 계획

Week2 일정 & 평가 기준

과정 안내

Week2 실습 일정 (5일)

시간	Day1	Day2	Day3	Day4	Day5
09:00	Cursor AI 설치·사용법	TDD_TV 실습	sHealth 실습	Feedback Analyzer 실습	Prompting BP 사례 공유
10:00			sHealth 발표		
11:00			UnitConverter 실습		
12:00	점심	점심	점심	점심	점심
13:00	Cursor AI 설치·사용법	TDD_TV 실습	UnitConverter 실습	Feedback Analyzer 실습	
14:00		sHealth 실습			
16:00					Gilded Rose 실습
17:00		Cursor AI 사용법	UnitConverter 발표	Feedback Analyzer 발표	

과정 평가 기준 (총 300점)

필기시험

실습 평가 × 6개

종합과제 + 필기

섹션 1

실습 회고

KPT 워크숍

Keep · Problem · Try 구조화 회고

KPT 회고 — Keep: 계속 유지할 것들

Keep 

Keep — C++ & Java 실습에서 잘 된 것들 (각자 포스트잇 작성 → 공유)

Q1 무엇이 잘 되었는가? — Cursor AI가 실제로 시간을 절약해 준 순간은?

Q2 어떤 관행을 계속 유지해야 하는가? — 팀에 도움이 된 프롬프트 패턴은?

Q3 팀에게 도움이 된 것들은? — @Codebase·@파일 컨텍스트 활용법?

코드 탐색

 @Codebase로 TDD_TV 전체 구조를 10분 만에 파악 → 기존 2~3시간 소요 대비 획기적 단축

 @Codebase로 C++ ITuner 인터페이스 의존성 그래프 자동 생성 → 구조 이해 가속

리팩토링 제안

 Composer(Ctrl+I)로 SRP 위반 TVController 분리 → 5분 만에 Service/Controller 구조 완성

 Cmd+K로 C++ God Class 분리 제안 → AgedBrielltem·SulfurasItem 클래스 분리 즉시 적용

테스트 자동화

 Chat에서 /test로 JUnit 5 경계값 테스트 자동 생성 → 커버리지 40%→80% 달성

 @FakeTuner.h 참조해 Google Test 케이스 자동 생성 → 반복 작업 90% 감소

문서화·보고서

 review_report.md 자동 생성 → IMRAD 구조 보고서를 5분 만에 초안 완성

 Doxygen 주석 일괄 추가 → 전체 파일 JSDoc 적용 시간 95% 절감

 Keep 항목은 팀 .cursorrules에 Best Practice로 등록하세요. 다음 프로젝트에서도 자동으로 적용됩니다.

Problem — 무엇이 팀을 느리게 했는가? 어떤 프로세스가 비효율적이었는가?

🌀 할루시네이션 & 검증 부담

- 🕒 존재하지 않는 Spring Boot API 생성 → 런타임 에러. 매번 실행 검증 필요.
- ⚙️ 없는 STL 메서드 참조 → 컴파일 에러. C++ 표준 버전 혼동(C++14/17).

📉 컨텍스트 손실

- 🕒 긴 대화 후 초기 요구사항 망각 → 엉뚱한 클래스 수정. /clear 후 재시작 빈번.
- ⚙️ 대용량 헤더 파일 참조 시 컨텍스트 초과 → 분할 참조 전략 필요.

🧑 프롬프트 엔지니어링 부족

- 🕒 "리뷰해줘" 수준의 모호한 프롬프트 → 표면적 분석만. P-C-T-F 미적용.
- ⚙️ C++17 특화 지시 없음 → C++11 패턴 생성. 버전·STL 명시 습관 미형성.

📄 Diff 검토 생략

- 🕒 Composer Accept All 남발 → 의도치 않은 파일 수정. 불필요한 import 추가.
- ⚙️ 헤더/소스 파일 동시 수정 시 누락 확인 안 함 → 링크 에러 발생.

Try — 다음에 다르게 해볼 것들 (구체적 실행 방안)

할루시네이션 → 생성 직후 즉시 빌드·테스트 실행을 프롬프트에 포함. "구현 후 빌드하고 에러 있으면 수정해줘"

컨텍스트 손실 → 대화 초반 @파일 명시적 참조. 30분마다 핵심 요구사항 재주입. CLAUDE.md/cursorrules 상시 로드

모호한 프롬프트 → P-C-T-F 4단계 + 7단계 구성법 적용. 출력 형식(Markdown 표·코드 블록)을 매번 명시

Diff 검토 생략 → Composer 사용 후 'Accept All' 금지 → 파일별 개별 Diff 확인 후 수락. 팀 규칙으로 문서화

💡 Try 항목은 다음 실습 시작 전 .cursorrules에 규칙으로 등록하세요. AI 행동이 자동으로 개선됩니다.

섹션 2

Prompting BP

우수 사례

C++ & Java 실전에서 검증된 Best Practice 패턴

BP 사례 1 — 코드 리뷰 & 아키텍처 설계 우수 프롬프트

BP #1

🔍 코드 리뷰 우수 사례 (Before → After 프롬프트)

❌ Before — 비효율적 프롬프트

모호한 프롬프트 (Java/C++ 공통)

"이 코드 리뷰해줘."
"TVController 좀 봐줘."
"뭐가 문제야?"

✅ After — P-C-T-F 구조화 프롬프트

Java / C++ 동일 P-C-T-F 패턴

[P] 당신은 10년 경력 시니어 Java 아키텍트입니다.
[C] @TVChannelController.java - TDD로 개발, JUnit 5, Java 21
[T] SOLID 원칙·코드 스멜·테스트 가능성 관점으로 리뷰하고
[F] Critical/Major/Minor 분류 + Before/After 코드를 Markdown 표로

아키텍처 설계 우수 사례 — C++ & Java 구조 설계 프롬프트

☕ Java

@Codebase UnitConverter를 OCP/SRP 원칙으로 설계해줘.

조건:

- LengthUnit 인터페이스: 새 단위 추가 시 기존 코드 수정 없음
 - InputParser, ConversionEngine, OutputFormatter 클래스 분리
 - units.json에서 비율 로드 (설정 외부화)
- 구현 후 "mvn test" 실행해서 에러 있으면 수정해줘.

⚙️ C++17

@Codebase C++17로 UnitConverter를 설계해줘.

조건:

- ILengthUnit 순수 가상 함수로 OCP 구현
 - unique_ptr<ILengthUnit>로 소유권 관리
 - nlohmann/json으로 외부 설정 로드
 - Google Test로 각 단위 변환 검증
- CMakeLists.txt 수정도 함께 알려줘.

95%

설계 초안 시간 단축

10분

PRD → 클래스 구조 완성

3배

리뷰 커버리지 향상

0건

설계 논의 중 구조 오류

BP 사례 2 — 성능 최적화 & 테스트 자동화 우수 프롬프트

BP #2

⚙️ C++ — BMI 성능 최적화

@BMICalculator.cpp 성능 최적화해줘:

1. `std::vector`→`std::unordered_map`으로
연령대 그룹핑 $O(n)$ 달성
 2. `std::transform` + `lambda`로
함수형 파이프라인 구성
 3. 파일 읽기: `getline` 대신 `mmap` 검토
(100만 건 기준 벤치마크 포함)
- C++17 병렬 알고리즘 (`std::execution::par`)
적용 가능성도 분석해줘.

⚙️ C++ — Google Test 자동화

@TVChannelController.h /test
Google Test 케이스를 작성해줘:

```
[구조]
TEST_F(TVControllerTest, [시나리오명]) {
    // Given: FakeTuner 초기 상태 설정
    // When: 버튼 입력 시뮬레이션
    // Then: EXPECT_EQ로 결과 검증
}
```

[필수 케이스]
- 숫자버튼 채널 변경 (경계값 포함)
- 선호채널 토글 (add/remove)
- 다음선호채널 wrap-around
ctest --output-on-failure로 실행해줘.

BP 사례 2 — 성능 최적화 & 테스트 자동화 우수 프롬프트

BP #2

⚡ 성능 최적화 우수 사례

Java - SHealth BMI 성능 분석

@SHealthAnalyzer.java @shealth.dat 성능 분석해줘:

1. 현재 $O(n^2)$ 루프를 Stream API로 최적화
 2. 나이대별 평균 계산 $\rightarrow$ Map<Integer, Double> 캐싱으로 중복 계산 제거
 3. 대용량 CSV (100만 건) 처리 시
BufferedReader로 메모리 최적화
- 각 개선 전/후 예상 성능 차이도 설명해줘.

🔧 테스트 자동화 우수 사례

Java - TDD_TV 테스트 자동화

@TVChannelController.java /test
다음 기준으로 JUnit 5 테스트 작성해줘:

[커버리지 목표] 90% 이상

[우선순위]

1. 경계값: 채널 0, 99, 두 자리 입력
2. 상태 전이: AwaitingFirst/SecondDigit
3. 예외: null 입력, ITuner 예외 발생 시
4. Mock: Mockito verify()로 setCH 호출 검증

FIRST 원칙 준수 여부도 체크해줘.

BP 사례 3 — CI/CD 자동화 & 배포·운영 우수 프롬프트

BP #3



CI/CD & 배포 자동화 우수 사례 (GitHub Actions 연동)



C++ CMake — CI 파이프라인

GitHub Actions C++ CI 워크플로우:

조건:

- C++17 + CMake + Google Test
- Ubuntu + Windows 멀티 플랫폼

.github/workflows/ci.yml 생성:

1. `cmake -B build -DCMAKE_BUILD_TYPE=Debug`
2. `cmake --build build`
3. `cd build && ctest --output-on-failure`
4. gcov/lcov로 커버리지 측정
5. 커버리지 리포트를 GitHub Pages 배포
Artifact로 테스트 결과 파일 저장해줘.



Java Maven — CI 파이프라인

GitHub Actions 워크플로우를 작성해줘:

조건:

- Java 21 + Maven
- Push/PR 시 자동 실행
- 단계: Build → Test → Coverage → PR 코멘트

.github/workflows/ci.yml 생성:

1. `mvn compile` (빌드 검증)
2. `mvn test` (JUnit 5 실행)
3. JaCoCo 커버리지 80% 미달 시 Fail
4. 테스트 결과를 PR 코멘트로 자동 게시
Cursor AI 분석 결과도 PR 코멘트에 포함해줘.

BP 사례 3 — CI/CD 자동화 & 배포·운영 우수 프롬프트

BP #3

AI 코드 리뷰 자동화 — Cursor AI × GitHub Actions 연동

.github/workflows/ai-review.yml — PR 자동 AI 리뷰

```
name: AI Code Review
on: [pull_request]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - name: Generate diff
        run: git diff origin/main...HEAD > changes.diff
      - name: AI Review via Cursor rules
        run: |
          # .cursorrules 기반 품질 게이트 자동 검사
          npm run lint && npm run typecheck          # Java: mvn checkstyle:check
          # 커버리지 임계값 80% 검증
          mvn test jacoco:check -Djacoco.minimum=0.80  # C++: cmake --build build && ctest
```



CI/CD 프롬프트 팁: 언어·빌드도구·플랫폼·임계값을 명시하면 실행 가능한 워크플로우가 바로 생성됩니다.

섹션 3

팀 액션 아이템 & SMART 목표

구체적·실행 가능한 개선 계획 수립

팀 차원 액션 아이템 & 개인 학습 성과 공유

액션 플랜

📋 팀 액션 아이템 추적 템플릿 (SMART 목표)

영역	구체적 액션	책임자	기한	성공 기준	상태
Prompting	.cursorrules 팀 공용 파일 작성 (P-C-T-F 규칙 포함)	팀장	1주	모든 PR에서 일관된 AI 응답 확인	🟡 진행
테스트 품질	커버리지 80% 미달 PR Fail 규칙 CI에 추가	DevOps 담당	2주	main 브랜치 커버리지 80%+ 유지	🟡 예정
코드 리뷰	Cursor AI 리뷰 → PR 코멘트 자동화 워크플로우	전체	3주	수동 리뷰 시간 50% 단축	🟡 예정
문서화	Javadoc/Doxygen 일괄 생성 스크립트 공유	팀원 A	1개월	전체 public API 문서화 100%	🟡 예정
현업 적용	소규모 레거시 모듈 1개 선정 → AI 리팩토링 시범 적용	팀 전체	3개월	코드 스멜 수 50%↓ / 테스트 커버리지 70%↑	🟡 예정

🌟 개인별 학습 성과 공유 항목 (6~7분 발표)

- 💡 새로 배운 기술·개념 — Cursor @Codebase 활용법 / C++ STL 최적화 패턴 / JUnit 5 경계값 테스트
- 📊 개인 역량 향상 부분 — 프롬프트 엔지니어링 / 코드 리뷰 관점 / TDD 사이클 속도
- 📚 추천 학습 자료 — cursor.directory (Rules 공유) / Anthropic Prompt Guide / Clean Code Korea

📌 발표 구조: ① 실습에서 가장 임팩트 있었던 AI 활용 순간 → ② Before/After 비교 → ③ 현업 적용 계획 → ④ 팀 공유 인사이트

섹션 4

현업 적용

로드맵

1개월 · 3개월 · 6개월 단계별 AI 내재화 계획

현업 적용 로드맵 & 회고 보고서 형식

로드맵

📖 현업 AI 내재화 타임라인



1개월

즉시 적용 — 도구 정착

- 팀 공용 .cursorrules 파일 완성 & Git 등록
- Cursor AI로 일일 코드 리뷰 습관 형성
- 기존 프로젝트 1개 모듈 @Codebase 분석
- Java: JUnit 5 커버리지 80% 목표 설정
- C++: Google Test + CMake CI 파이프라인 구축



3개월

확장 — 팀 프로세스 통합

- PR 자동 AI 리뷰 워크플로우 운영 안정화
- 레거시 모듈 1~2개 AI 지원 리팩토링 완료
- 팀 Prompting BP 가이드 문서 v1.0 발행
- 커버리지 80%+ 유지 CI 게이트 전체 적용
- 사내 AI 도구 활용 세미나 발표 1회



6개월

내재화 — 조직 문화 정착

- AI 코드 품질 지표 대시보드 구축 (커버리지·스멜 수)
- 신규 팀원 온보딩: AI 활용 가이드 포함
- MCP 연동: GitHub Issues→코드→PR 자동화 운영
- 팀 AI 활용 성과 측정 (개발 시간 단축률 정량화)
- 사내 Best Practice 발표 & 타 팀 전파



최초 제출 회고 보고서 형식 (Markdown)

Prompting BP 회고 ## 1. Keep ## 2. Problem ## 3. Try ## 4. 개인별 학습 성과 ## 5. 팀 액션 아이템 (표) ## 6. 현업 적용 계획

📎 제출물: Prompting_BP_xx 레포지토리에 report.md 커밋 + GitHub PR 생성 + 리뷰어 등록.

워크숍 진행 가이드 — 50분 구조화 세션

워크숍 가이드

10분

준비 & 체크인

- ESVP 체크 (Explorer·Shopper·Vacationer·Prisoner)
- 실습 3주 여정 감정 온도계 (🟢 좋음 🟡 보통 🔴 어려움)
- 시간축 작성: 1주차(TDD/리팩토링) → 2주차(Cursor AI) → 3주차(팀 프로젝트)

10분

데이터 수집

- 4L 매트릭스 작성 (포스트잇·개인 작성 → 벽 게시)
- Learned(배운것)·Lacked(부족한것)·Liked(좋았던것)·Longed for(바라는것)
- Java/C++ 각 언어별 Cursor AI 활용 경험 색깔 구분 (🟣 / 🌸)

10분

통찰 도출

- 유사 항목 군집화 → 브레인스토밍
- 점투표로 우선순위 설정 (각자 3개 점스티커)
- 생선가시 다이어그램: "왜 Cursor AI 활용이 효과적이었나?"

10분

SMART 액션 플랜

- 팀 공용 .cursorrules 작성 계획 → 책임자·기한 배정
- 개인별 3개월 학습 계획 카드 작성 (현업 적용 포함)
- 다음 실습/프로젝트 개선 규칙 3가지 선정

20분

발표 & 마무리

- 팀별 6~7분 발표: Keep 2개 + Problem 2개 + Try 2개
- Best Practice 공유: Java·C++ 각 1개씩 실전 프롬프트 시연
- 강사 피드백 + 다음 과정(Gilded Rose) 연결

🚩 발표 규칙: 각 팀 6~7분. 반드시 Java/C++ 코드 예시 포함. Cursor Chat 실시간 시연 환영. 우수 사례는 팀 레포지토리에 등록.

좋은 프롬프트는 좋은 질문에서 시작한다. 코드도, AI도, 성장도.

- ✓ Keep: .cursorrules에 Best Practice를 등록하면 팀 전체의 AI가 개선된다
- ✓ Problem: 모호한 프롬프트는 P-C-T-F + 실행 검증 요청으로 해결할 수 있다
- ✓ Try: 단계적 현업 적용 — 1개월(도구 정착) → 3개월(팀 통합) → 6개월(문화 내재화)
- ✓ C++ & Java: 언어·버전·빌드도구를 명시한 프롬프트가 가장 효과적이다

수 고 하 셧 습 니 다 🎉